

FitBuddy

AI Fitness Plan Generator using Gemini Models

Personalized Fitness Planning

Workout, recovery and nutrition support



Mon	Workout
Tue	Cardio
Wed	Strength
Thu	Recovery
Fri	Workout

Project Documentation

FastAPI • Google Gemini AI • SQLite • Jinja2

1. Introduction

FitBuddy is a web-based application that uses artificial intelligence to generate personalized workout plans and nutrition or recovery tips. The application accepts user information such as age, weight, fitness goal and preferred workout intensity, then creates a customized 7-day fitness plan.

FitBuddy is built using FastAPI and integrated with Google Gemini models. SQLite with SQLAlchemy is used to store user information and generated plans, while HTML and Jinja2 provide the interactive web interface.

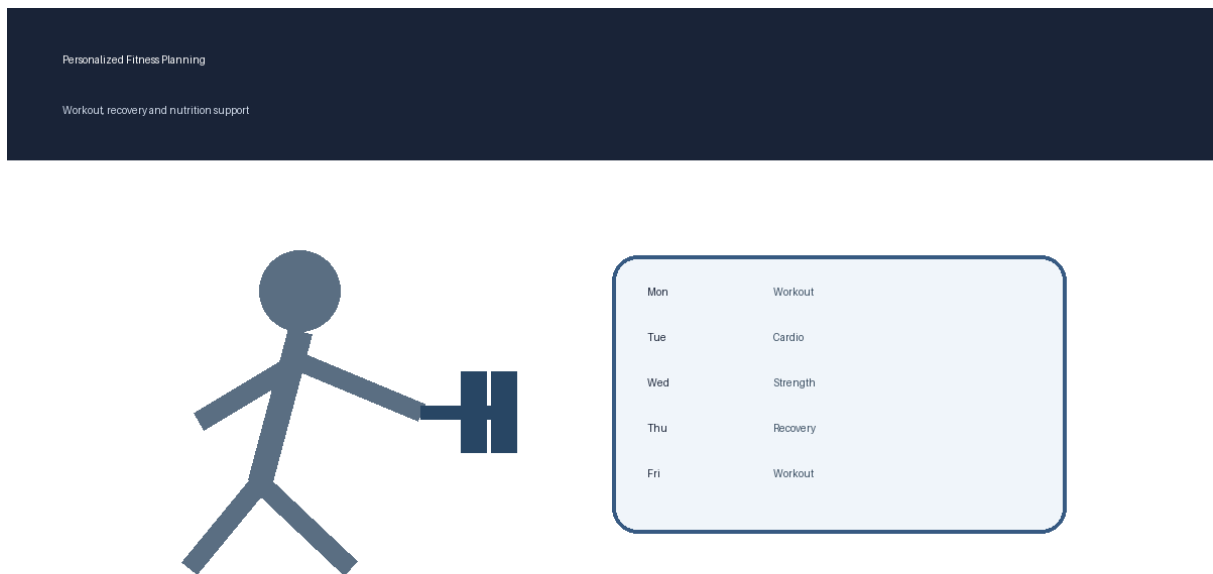


Figure 1. Conceptual view of personalized fitness planning.

2. Objectives

- Generate structured 7-day workout plans based on user goals and intensity.
- Provide concise and practical nutrition or recovery tips.
- Allow users to submit feedback and regenerate an updated workout plan.
- Store user information and plans in an SQLite database.
- Provide an administrative view of registered users and their plans.

3. System Architecture

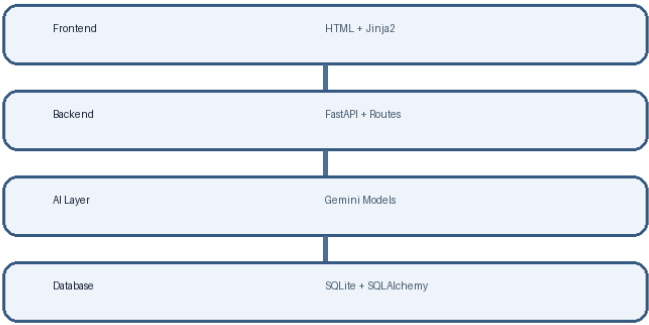


Figure 2. FitBuddy technical architecture.

The application follows a modular architecture consisting of a frontend, FastAPI backend, Gemini AI layer and SQLite database. The frontend collects information and displays results; FastAPI manages routing and application logic; Gemini generates plans and tips; and SQLite stores user and plan data.

4. Technologies Used

Technology	Purpose
Python	Main programming language
FastAPI	Backend framework and API routing
Google Gemini	AI-based workout and nutrition generation
Jinja2	Dynamic HTML template rendering
SQLite	Local data storage
SQLAlchemy	Database ORM
Uvicorn	ASGI development server
HTML/CSS	User interface

5. Main Features

Workout Plan Generation: The system uses user goal and intensity to generate a structured 7-day plan containing warm-up, main workout, exercise details, sets/repetitions and cooldown or recovery guidance.

Nutrition and Recovery Tips: A fast AI response provides practical guidance such as hydration, protein suggestions or recovery practices.

Feedback-Based Updating: Users can request changes such as more cardio or additional rest days. The original plan and feedback are passed to the AI to create a revised plan.

User and Plan Storage: User details and generated plans are stored in SQLite. Updated plans are stored separately so the original version is preserved.

Admin View: The admin dashboard displays registered users and their original and updated plans.

6. Project Workflow



Figure 3. FitBuddy workflow.

1. User enters personal details and fitness preferences. 2. FastAPI receives and validates the input. 3. The appropriate Gemini model processes the request. 4. A personalized plan and nutrition/recovery tip are returned. 5. The result is displayed through the Jinja2 frontend. 6. Feedback can be submitted to update the plan. 7. User and plan information is stored in SQLite.

7. Gemini Model Integration

According to the project document, Gemini 1.5 Pro is used for structured 7-day workout generation and plan updating, while Gemini Flash is used for fast nutrition tips. The documented reasons for choosing Gemini include API-based integration with FastAPI, support for structured formatting and fast feedback-based revisions.

8. Backend Routes

/ – Displays the home page and user input form.

/generate-workout – Receives user input, generates the workout and nutrition tip, saves the information, and renders the result.

/submit-feedback – Retrieves the existing plan, sends the original plan and feedback to Gemini, stores the revised plan, and displays the update.

/view-all-users – Displays the admin view containing users and their plans.

9. Frontend

The frontend uses Jinja2-powered HTML templates. The documented templates include `index.html` for user input, `result.html` for displaying the generated plan and nutrition tip, and `all_users.html` for the admin panel. The interface is designed to be responsive using Flexbox, media queries, consistent buttons, rounded result blocks and a dark fitness-oriented visual style.

10. Database

SQLite with SQLAlchemy ORM is used for persistence. The stored user information includes user ID, name, age, weight, goal and intensity. AI-generated plans are also stored, and feedback-based updates are maintained separately to preserve both original and updated versions.

11. Project Structure

```
FitBuddy/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── database.py
│   ├── schemas.py
│   ├── ai.py
│   ├── routes.py
│   └── templates/
│       ├── index.html
│       ├── result.html
│       ├── admin_login.html
│       └── all_users.html
├── static/
│   └── css/style.css
├── tests/
│   └── test_app.py
├── requirements.txt
├── .env.example
└── .gitignore
```

12. Installation and Execution

Step 1: Create a Python virtual environment using `python -m venv venv`.

Step 2: On Windows, activate it using `venv\Scripts\activate`.

Step 3: Install dependencies using `pip install -r requirements.txt`.

Step 4: Configure the Gemini API key through the environment configuration.

Step 5: Start the FastAPI server using `uvicorn app.main:app --reload`.

Step 6: Open the application at `http://127.0.0.1:8000`. The FastAPI interactive documentation is available at `/docs`.

13. Testing

The project documentation describes local deployment verification by starting the Uvicorn server and accessing the web application through a browser. The `/docs` endpoint can be used to explore and test API endpoints. Functional testing should cover user input, workout generation, nutrition tips, feedback-based updates, database storage and the admin view.

14. Expected Output

A successful run provides a web interface where a user enters fitness information and receives a personalized 7-day workout plan together with a nutrition or recovery tip. The user can then submit feedback and receive an updated plan. Administrators can view stored users and their plans through the admin interface.

15. Advantages

- Personalized fitness planning using AI.
- Structured day-wise workout output.
- Dynamic plan refinement through feedback.
- Database-backed storage of users and plans.
- Web-based interface accessible through a browser.
- Modular architecture that separates frontend, backend, AI and database responsibilities.

16. Future Enhancements

The documented project prepares the application for future deployment on cloud platforms. Possible future development can include richer progress tracking, authentication improvements, expanded fitness inputs and additional integrations, provided these are added to the implementation.

17. Conclusion

FitBuddy demonstrates how generative AI can be integrated into a FastAPI web application to create personalized fitness plans. By combining Gemini models with a structured backend, Jinja2 interface and SQLite persistence, the project provides workout generation, nutrition guidance, feedback-based plan updates and administrative viewing in one application.

18. References / Project Basis

This documentation is based on the supplied FitBuddy project document, including its project description, architecture, workflow, model selection, backend routes, frontend development and deployment instructions. ■filecite is not embedded in the generated PDF; the source is the user-provided project document.